Data-gap identification

Revised 2026-08-11. Supersedes the 2026-08-07 edition.

One symptom — **"no numbers came back"** — has four different causes. Telling them apart is the whole problem. Confusing any two produces a wrong answer, and each confusion is wrong in its own direction.

The four verdicts

Verdict	What happened	Specialist does	Scored as
RESOLVED	The name was off, but it points at something real	proceeds — nothing to report	success
MISTAKE	The name matches several things, or none, but something close exists	fix the name, re-issue	failure, retryable
GAP	Absent, or present-but-blank	record in `data_gaps`, move on	success — the tool worked
REAL NEGATIVE	The data is all there; the answer is genuinely "none"	report it as a finding	success

`GAP` and `REAL NEGATIVE` are both true answers and must never be retried. `MISTAKE` is the only one worth another call.

The decision

```
a table or column was asked for, and nothing came back
■
■■ 1. Does the name RESOLVE?                          [strict test]
■     exact → catalog alias → naming convention → normalized → unique near-miss
■     ■■ yes ..................................... RESOLVED  (proceed)
■
■■ 2. Does anything in this case RESEMBLE it?       [loose test]
■     ■■ yes ..................................... MISTAKE   (name the candidates)
■
■■ 3. Nothing resembles it ....................... GAP       (absent)
■
■■ 4. It resolved and exists — what came back?
      ■■ values ................................... ANSWER
      ■■ rows matched, every value blank ............ GAP       (empty)
      ■■ no rows matched the filter ................. REAL NEGATIVE
```

Steps 1 and 2 ask about resemblance with **different strictness**, and the gap between them is deliberate — it is where a name lands when it is recognisably close to something but not close enough to act on unasked.

Step 1 is the important one: **the system tries to make the call work before it reports anything missing.** `model_scores_transaction` → `modelling_data_transaction` via the catalog alias; `modelling_dat` → `modelling_data` and `SBFE_Scoree` → `SBFE Score` as unique near-misses. Corrections are logged (`table_name_autocorrected`, `column_name_autocorrected`) and the resolved name appears in the output, so nothing is silently misattributed.

What "resemble" means

Not a feeling — two predicates, and they are deliberately different. Both normalize first: lowercase, drop non-alphanumerics, drop trailing digits.

To BIND (step 1 — resolve it and proceed, saying nothing). Strict. Either test qualifies, and the result must be the *only* candidate:

- **one edit** — Damerau-Levenshtein ≤ 1: a character added, dropped, **substituted** or **transposed**. `modeling_data` → `modelling_data`, which matters because the specialist is named `modeling` while the table is `modelling`.
- **containment ≥ 70%** — one name contains the other and the shorter is at least 70% of the longer. Catches truncations of several characters that an edit budget of 1 would not: `model_scores_trans` → `model_scores_transaction`.

Plus two floors: names under 4 characters never bind, and two equally good candidates never bind — `score_a` and `score_b` are both one edit from `score_c`, so the resolver declines rather than picking. That refusal is what makes one-edit matching safe.

To SUGGEST (step 2 — name the candidates in the error). Loose: plain containment, no ratio, no floor.

The gap between the two is not slack, it is the point:

	binds?	suggests?	outcome
`oop_interaction`	yes — 82% of the column	—	RESOLVED, answers 35.18
`oop`	no — 3 chars, 18%	yes → `oop_interaction_max`	MISTAKE, candidate named
`income`	no	no	GAP

`oop` is the case worth understanding. Binding it would answer `max(oop_interaction_max) = 35.18` for a question about the OOP *concept* — attributing one variable's number to a broader idea. But if the loose tier did not catch it either, `oop` would fall through to "nothing resembles it" and be reported as **a data gap, while the column sits right there**. So the system declines to answer and points at it instead.

Why each confusion costs something

Confusion	What it does
GAP read as MISTAKE	The specialist retries a call that can never succeed. Case 11854808010: six ca an absent table, then the whole retry attempt on the same six.
MISTAKE read as GAP	A real variable is abandoned over a spelling slip, and the reviewer is told data i missing when it is not.
ABSENT read as EMPTY	"The column is blank for this case" — about a column that isn't there. Both onc returned the identical string.
REAL NEGATIVE read as GAP	"We couldn't check" instead of "we checked and there are none" — the weaker claim, and the wrong one.
GAP read as failure	An honest report is scored as a broken run and quarantined; the specialist that the truth is discarded.

That last one is why `DATA GAP:` is checked **first** in `grounding.classify_tool_output` and returns "not a failure". Everything else about a run being flagged follows from that: a flagged call sets `ungrounded`, triggers a retry, and if unresolved raises `_SkipPersistence` — the run writes nothing to the KB, Amem or charts.

Prevention beats detection

The best gap is the one the specialist never chases. On its first round it receives an inventory of the case's real columns, which now also states:

```
NOT IN THIS CASE: model_scores_transaction, score_drivers_transaction
  …this case's data does NOT include them… do not try, and do not retry
  after a failure. Their absence is the SHAPE OF THIS CASE'S DATA, not a
  system fault…
```

Silence is not a signal. The inventory used to simply omit absent tables, which an LLM reads as "not shown here" rather than "absent".

When the specialist misreads a correct result

Tool-level correctness is not enough — the answer can still deny what the tool returned.

- `absence_contradicted_by_rows` — the answer says "none / zero / no records" while no call in the run returned 0. Triggers an `absence_reread`: the transcript it already has, plus "read the count off it literally". It does not re-query; the number is already in front of it.
- **The synthesis gate** — before any answer path, a null must be classified ABSENT / UNTESTED / EMPTY / REAL NEGATIVE. A null that contradicts a curated report is not publishable.

Worked example — case 11854808010

The `modeling` skill names `model_scores_transaction` a dozen times. That case has no transaction-level CSV; it stops at month grain.

Before. The inventory omitted the table silently. Six `summarize_by_group` calls at it, six "table not found", the entire retry attempt spent on the same six, and then published: *"No access to transaction-level modeling data (`model_scores_transaction`), which precludes analysis of linked or recurrent high-risk events."* True — and it reads like a broken system. Every one of those calls was scored a failure, putting the run at risk of quarantine for being right.

After. The inventory names the absence up front. If a call is made anyway it returns a `DATA GAP:` — a success, not a failure — and lists the tables the case does have. On case 366132845011, which HAS the table, nothing changes: the alias resolves it as it always did.

Limits

- `absence_reread` **catches more than it fixes** — 9 fired, 3 corrected. When the re-read fails the answer ships anyway; the check detects, it does not block.
- **Absence is inferred from this case's export**, never from a statement about what the case ought to contain. A table missing because an upstream feed failed looks identical to one the case never had.
- **Ambiguous near-misses refuse rather than guess**, so some genuine typos still surface as misses.